



**Infraestrutura e Cloud Computing com Docker,
Kubernetes e AWS**
Autoria: Jacqueline Oliveira

Introdução

A disciplina de **Infraestrutura e Cloud Computing com Docker, Kubernetes e AWS** foi concebida com o propósito de preparar profissionais para atuar de forma prática e estratégica em ambientes de desenvolvimento modernos e altamente escaláveis. A proposta não é apenas apresentar ferramentas, mas sim construir, ao longo das aulas, uma compreensão sólida e aplicada dos principais fundamentos e tecnologias que sustentam a engenharia de software em nuvem.

Ao longo da disciplina, percorremos uma jornada que começa com a base da infraestrutura moderna: o entendimento do que é cloud computing e por que esse modelo transformou a forma como construímos e entregamos software. Falamos sobre conceitos essenciais como escalabilidade, resiliência, disponibilidade e modelos de serviço como IaaS, PaaS e SaaS, sempre contextualizados dentro do universo corporativo.

Na sequência, mergulhamos em **Docker**, que é hoje uma das tecnologias mais relevantes para o empacotamento e a execução de aplicações. Através dele, aprendemos como isolar ambientes, automatizar builds, otimizar imagens e criar containers reutilizáveis, garantindo que a aplicação funcione da mesma forma em qualquer máquina. Trabalhamos não apenas com fundamentos, mas também com boas práticas e estratégias de versionamento e deploy.

Para complementar essa abordagem, estudamos também o **Podman**, que surge como uma alternativa ao Docker, oferecendo compatibilidade com o mesmo ecossistema, porém com uma arquitetura mais leve e segura. A ideia foi apresentar diferentes possibilidades, permitindo que o profissional escolha a ferramenta mais adequada para o seu contexto.

Com as aplicações já containerizadas, avançamos para o **Kubernetes**, a plataforma de orquestração de containers que se tornou padrão na indústria. Com ele, exploramos conceitos como pods, services, deployments, escalabilidade automática e monitoramento de saúde. Criamos nossos próprios clusters e compreendemos como o Kubernetes gerencia aplicações distribuídas de forma eficiente.

A segunda metade da disciplina foi dedicada à **Amazon Web Services (AWS)**, uma das principais plataformas de nuvem do mundo. Começamos pelos fundamentos e, gradualmente, integramos nossa aplicação aos principais serviços da AWS: armazenamento de arquivos com o S3, persistência de dados com o DynamoDB, execução de funções com o Lambda e automação de deploys com o GitHub Actions e Jenkins. Cada uma dessas integrações foi feita com foco na prática e no uso real das ferramentas.

Um ponto central ao longo de toda a disciplina foi a construção incremental de uma aplicação real — o UniPDI — que serviu de base para todas as implementações. Esse projeto não apenas deu sentido às tecnologias apresentadas, como também permitiu que os alunos visualizassem o impacto de cada nova etapa da arquitetura.

Ao final da disciplina, os alunos não apenas dominaram ferramentas essenciais do ecossistema DevOps e cloud, mas também compreenderam como integrá-las em fluxos de desenvolvimento contínuo, com foco em escalabilidade, segurança, eficiência e boas práticas.

Essa formação é essencial para profissionais que desejam atuar em ambientes corporativos modernos, que exigem agilidade, automação e profundo conhecimento técnico. Mais do que ensinar ferramentas, esta disciplina promoveu uma mentalidade voltada à resolução de problemas reais, com autonomia, responsabilidade e visão de arquitetura.

Capítulo 1: Introdução a Infra e Cloud Computing

Seja bem-vindo à nossa disciplina de Infraestrutura e Cloud Computing. Eu sou a Jaqueline Oliveira e vou acompanhar você nessa jornada. Talvez você esteja se perguntando: se essa área não é minha especialização principal, como posso ensinar esse assunto? A resposta é simples: essa disciplina é pensada sob a perspectiva de um desenvolvedor. O foco é mostrar o que nós, como desenvolvedores back-end, precisamos saber para lidar com infraestrutura e computação em nuvem no nosso dia a dia.

A minha formação é em Ciência da Computação, com pós-graduação em Arquitetura e Engenharia de Software. Desde 2010 atuo como engenheira de software, e, a partir de 2020, comecei também a compartilhar conhecimento como professora de Tecnologia. Gosto dessa troca com a comunidade, e você vai me encontrar no Discord para tirar dúvidas, responder enquetes e conversar sobre os temas abordados aqui.

Começamos com uma definição que gosto muito sobre Cloud Computing: "conjunto de recursos customizáveis como redes, servidores, armazenamento, aplicações e serviços". Essa definição reflete exatamente a realidade que vivi em todas as empresas por onde passei. Não existe uma configuração única. Cada empresa escolhe o conjunto de ferramentas e serviços que mais se adequa à sua necessidade. Uma pode usar AWS, outra Azure, ou Google Cloud. O banco de dados pode ser NoSQL em um lugar e relacional em outro. Isso torna a área extremamente customizável e, ao mesmo tempo, complexa.

O que mais me fascina na computação em nuvem é como ela transformou profundamente a forma como interagimos com tecnologia. Essa mudança não atingiu apenas os profissionais da área, mas qualquer pessoa conectada à internet. Costumo comparar a realidade da minha geração com a da minha filha de 14 anos. Hoje ela escuta música em qualquer lugar, monta playlists e compartilha com facilidade. Na minha época, eu gravava músicas do rádio em uma fita cassete e usava um Walkman para ouvir. Essa diferença evidencia o quanto a nuvem mudou nosso modo de armazenar, processar e compartilhar informações.

Durante muito tempo, a infraestrutura era toda local. As empresas mantinham servidores, armazenamento e rede dentro da própria estrutura, sendo

responsáveis por tudo: energia, refrigeração, segurança, atualizações. Esse modelo, conhecido como on-premise, era comum, mas apresentava riscos significativos. Me recordo de um cliente, um escritório de contabilidade, que teve todos os dados roubados junto com os servidores e backups. A perda foi total. Hoje, com a nuvem, é possível contar com backup automático, redundância e segurança de dados, mesmo em situações de roubo ou falhas.

Empresas passaram a adotar soluções como AWS, Azure, Google Cloud e Oracle porque essas plataformas oferecem muito mais que hospedagem. Elas entregam infraestrutura sob demanda. Isso significa que a aplicação pode crescer conforme a demanda, com escalabilidade, resiliência e segurança. Temos acesso a backup, mensageria, serviços gerenciados, opções de banco de dados e processamento de dados em larga escala.

A computação em nuvem se tornou a base da tecnologia moderna. Hoje, qualquer aplicação que desenvolvemos precisa considerar escalabilidade, disponibilidade e segurança. Por isso, mesmo como desenvolvedores, precisamos dominar conceitos de infraestrutura. Mas até onde vai esse conhecimento? O que é realmente essencial?

Para responder a essa pergunta, analisei diversas vagas de mercado. Nelas, identifiquei termos que aparecem com frequência: AWS, EC2, ECS, RDS, DynamoDB, Docker, Kubernetes, ferramentas de CI/CD, GitHub Actions, pipelines, mensageria como Kafka, SQS e SNS. É claro que não existe uma receita única, mas esses elementos são recorrentes.

Com base nisso, estruturei a ementa da disciplina em quatro grandes grupos:

1. **Docker**: construção e otimização de imagens;
2. **Kubernetes**: introdução ao orquestrador de containers;
3. **AWS para desenvolvedores Java**: uso de serviços como S3, Lambda e DynamoDB;
4. **CI/CD**: integração e entrega contínua com foco em automação.

Desde o início, deixo claro que infraestrutura e cloud computing por si só já seriam temas suficientes para uma nova pós-graduação. Portanto, não será

possível explorar todos os tópicos em profundidade. A proposta é trabalhar mão na massa com um projeto base, de forma prática, e usar o Discord para trocas, indicações de materiais complementares e eventos extras, conforme as necessidades da turma.

Nosso projeto base se chama **UnipDI**. Ele é simples e nos acompanha ao longo da disciplina. É composto por um banco de dados MongoDB, uma API com Spring Web e um front-end com Vite e React. O objetivo é criar PDIs (Planos de Desenvolvimento Individual). Através desse projeto, vamos praticar containerização, uso de serviços em nuvem e ferramentas de CI/CD. Vocês terão acesso ao repositório no GitHub e a todo material produzido.

A ideia é que vocês acompanhem o que eu fizer, pratiquem junto e tragam dúvidas. Não consigo cobrir todos os cenários de mercado, mas com a participação de vocês, consigo direcionar para os assuntos que fizerem mais sentido para o grupo. Assim, podemos aproveitar ao máximo essa experiência.

Então, seguimos juntos. No próximo encontro, vamos iniciar a parte prática. Até lá!

Capítulo 2: Fundamentos Docker

Dando continuidade aos nossos estudos, entramos agora em um dos principais tópicos que fazem parte da rotina de qualquer pessoa desenvolvedora: o Docker. Como em toda ferramenta que estudamos, gosto de começar pela raiz do problema. Afinal, antes de sairmos adotando soluções, precisamos entender claramente qual problema elas vieram resolver. Existe uma frase no Scrum que gosto muito: "ame o problema". Compreender o problema é o que nos torna melhores em encontrar soluções simples e eficazes.

Vamos então relembrar o contexto anterior ao surgimento do Docker. Quando ainda predominava o modelo on-premise, as empresas mantinham toda a infraestrutura localmente: servidores, rede, armazenamento, segurança, tudo sob responsabilidade da própria organização. Trabalhei em ambientes assim, inclusive em salas climatizadas próximas de zero grau, destinadas apenas aos servidores. O custo era altíssimo: equipamentos, energia, equipes de manutenção, escalabilidade limitada e, muitas vezes, desperdício de recursos. Além disso, havia o desafio de como distribuir aplicações com diferentes dependências em um mesmo ambiente. O uso inadequado do hardware causava conflitos de versão, de porta ou sobrecarga nos recursos.

Como solução intermediária, surgiram as máquinas virtuais. Através de um hipervisor, podíamos rodar vários sistemas operacionais sobre um mesmo host. Era um avanço: permitia isolamento, alocação específica de CPU, memória e armazenamento para cada VM, resolvendo em parte o problema de conflitos. Porém, não era a solução ideal. Cada máquina virtual exigia a instalação de um sistema operacional completo. Isso significava desperdício de processamento, espaço e memória. Atualizar cada SO era mais um ponto de manutenção, além do consumo elevado de recursos.

Foi nesse contexto que os containers surgiram como a evolução natural. A principal diferença é que os containers compartilham o mesmo sistema operacional do host, mas continuam operando de forma isolada. Isso elimina a necessidade de instalar um SO por instância e torna a execução mais leve, rápida e padronizada. Assim nasceu o Docker, a plataforma que popularizou o uso de containers.

O Docker permite empacotar aplicações com todas as suas dependências em um ambiente isolado e reproduzível. Isso resolve o clássico problema de "na minha máquina funciona". Ao utilizar containers, garantimos que a aplicação rodará da mesma forma em qualquer ambiente, seja no notebook do desenvolvedor, no servidor de testes ou em produção na nuvem. Além disso, facilita o compartilhamento do ambiente com colegas de equipe e torna os testes mais confiáveis.

Para utilizar Docker, é preciso instalar o Docker Desktop, disponível para Windows, Linux e Mac. Para fins didáticos, escolhi o Windows, por ser amplamente utilizado. Assim, conseguimos trabalhar tanto com a interface visual quanto com os comandos em terminal. Uma vez instalado, é possível gerenciar imagens, containers e volumes, além de visualizar logs e configurar execuções.

Um dos primeiros comandos que costumo usar é o `docker run` com a imagem `hello-world`. Ao executar esse comando, o Docker verifica se a imagem está presente localmente. Se não estiver, ele faz o *pull* do repositório (geralmente do Docker Hub) e executa o container. A mensagem de sucesso confirma que o ambiente está configurado corretamente. Isso também permite observar a diferença entre imagem e container: a imagem é como um modelo, um template, enquanto o container é uma instância em execução dessa imagem.

No Docker Desktop, é possível ver a lista de imagens baixadas e os containers criados, mesmo que estejam parados. Ao executar o comando `docker ps`, vemos os containers ativos. Um container baseado em um script simples, como o `hello-world`, é finalizado imediatamente após a execução, pois não há uma aplicação persistente rodando.

No nosso projeto UnipDI, utilizamos o MongoDB como banco de dados. Em vez de instalar o Mongo diretamente na máquina, optamos por utilizar um container com a imagem oficial do Mongo. Para isso, usamos o Docker Desktop ou o terminal para buscar a imagem no Docker Hub, fazer o *pull* e executar o container. Durante o processo, é possível definir portas, volumes e variáveis de ambiente. Por exemplo, mapeamos a porta padrão do Mongo (27017) do container para a máquina host, permitindo que a aplicação back-end se conecte normalmente ao banco.

Um ponto importante: containers são voláteis. Isso significa que, ao desligar um container, os dados internos são perdidos, a menos que configuremos volumes persistentes. Para ambientes de desenvolvimento e produção, isso é essencial. No nosso exemplo, ao criar um container Mongo sem volume, os dados eram perdidos ao reiniciar. Com o volume configurado, os dados se mantêm entre execuções.

Conforme o projeto evolui, surgem outras necessidades: executar simultaneamente o Mongo, o back-end e o front-end. Gerenciar isso com comandos individuais se torna complexo. É nesse momento que o docker-compose entra em cena. O docker-compose permite orquestrar vários containers a partir de um arquivo YAML. Nesse arquivo, definimos serviços, imagens, variáveis, portas e volumes. Com um simples comando `docker compose up`, todos os serviços são iniciados conforme especificado.

No repositório do projeto UnipDI, forneci um `docker-compose.yml` já configurado. Ao executar `docker compose up`, o Docker faz o pull das imagens, sobe os containers e conecta os serviços entre si. Isso significa que conseguimos iniciar a aplicação completa sem abrir IDEs ou terminais separados. A produtividade aumenta significativamente, especialmente em equipes de desenvolvimento e em ambientes de testes.

O sistema de camadas do Docker também contribui para sua eficiência. Ao construir imagens, o Docker reaproveita camadas existentes, tornando o processo mais rápido e leve. Esse conceito será importante quando criarmos nossas próprias imagens a partir de um `Dockerfile`, o que veremos nas próximas aulas.

Resumindo: o Docker revolucionou a forma como construímos, testamos e executamos aplicações. Ele simplifica o ambiente de desenvolvimento, aumenta a consistência entre ambientes e permite escalar serviços com facilidade. É uma ferramenta essencial para qualquer pessoa que trabalha com software atualmente. E é por isso que vamos explorá-la em profundidade ao longo das próximas aulas.

Capítulo 3: Criando e Disponibilizando Imagens

Agora que entendemos o funcionamento dos containers e utilizamos imagens prontas como a do MongoDB, chegou o momento de criarmos a nossa própria imagem da aplicação e disponibilizá-la no Docker Hub. Isso permite que qualquer

pessoa possa executar a aplicação sem precisar compilar ou configurar dependências localmente.

No meu Docker Hub, ainda não havia nenhum repositório. O objetivo era justamente criar uma imagem da aplicação, subir para esse repositório e permitir que vocês também a utilizem. O ponto de partida para isso é o Dockerfile, que é o arquivo onde definimos como a imagem será criada. Diferente do docker-compose, que serve para orquestrar múltiplos containers, o Dockerfile é uma espécie de receita que orienta o Docker na construção da imagem.

No IntelliJ, criei um novo arquivo Dockerfile na raiz do projeto back-end. A primeira instrução que definimos é o FROM, que indica qual imagem base será usada. Como nossa aplicação gera um arquivo JAR, precisamos de uma distribuição da JDK. Escolhi a eclipse-temurin com versão 21-alpine por ser uma opção leve e compatível com a versão do projeto.

Em seguida, defini um diretório de trabalho (WORKDIR) dentro do container, onde os arquivos serão copiados e executados. Utilizei /app como caminho padrão. Depois, incluí o comando COPY para transferir o JAR gerado na pasta target para esse diretório, renomeando-o para app.jar. Por fim, adicionei um ENTRYPOINT com o comando `java -jar /app/app.jar`, que será executado sempre que o container iniciar.

Outro ponto importante é o EXPOSE, que define qual porta será exposta pelo container. Nossa aplicação roda na porta 8080, então esse foi o valor utilizado.

Com o Dockerfile pronto, precisei gerar o JAR da aplicação. Isso foi feito via terminal com o comando `mvn clean package`. Depois disso, executei o comando `docker build -t unipdi-backend .` para construir a imagem. O processo criou a imagem com base nas instruções do Dockerfile e a armazenou localmente.

Verifiquei a presença da imagem tanto via terminal quanto pela interface do IntelliJ e do Docker Desktop. A imagem estava lá, criada com sucesso, pronta para ser transformada em container. Para isso, utilizei o comando `docker run -p 8080:8080 unipdi-backend`, que mapeia a porta interna do container para a porta 8080 da máquina local.

Entretanto, a aplicação precisava se conectar ao banco de dados MongoDB, que também estava rodando em container. Como containers isolados não se veem automaticamente, precisei passar uma variável de ambiente indicando a URI correta do Mongo. Essa URI usa um host especial interno do Docker, chamado `host.docker.internal`, e ficou assim: `mongodb://host.docker.internal:27017/unipdi`. Essa informação foi passada como variável de ambiente com o nome `SPRING_DATA_MONGODB_URI`.

Com isso, a aplicação iniciou corretamente, conectando-se ao banco e expondo a API na porta esperada. Foi possível acessar os dados via navegador e confirmar que tudo estava funcionando.

O próximo passo foi subir a imagem criada para o Docker Hub. Para isso, utilizei a função de *push* da IDE, selecionando o destino como Docker Registry e nomeando a imagem como `jacktech/unipdi-backend` com a tag `latest`. A imagem foi enviada para minha conta no Docker Hub e passou a ficar disponível publicamente.

A partir de agora, qualquer pessoa pode executar essa imagem diretamente com o comando `docker run jacktech/unipdi-backend`, mapeando as portas e configurando as variáveis conforme demonstrado. Isso facilita imensamente o compartilhamento da aplicação e acelera o processo de testes e execução.

Esse processo mostra o quanto é vantajoso empacotar aplicações em containers. E mais ainda: o quanto é vantajoso contar com arquivos como o `docker-compose.yml`, que automatiza a execução conjunta de múltiplos containers, evita comandos extensos e torna a execução mais fácil e reproduzível. Veremos isso com mais detalhes nas próximas aulas.

Por ora, o foco foi criar nossa imagem, testar sua execução, configurar corretamente a comunicação com outros containers e disponibilizá-la para uso externo. Espero que você também consiga fazer o mesmo com suas aplicações. Nos vemos na próxima aula!

Capítulo 4: Otimização de Imagens Docker

Neste ponto da disciplina, já conseguimos empacotar aplicações em imagens Docker e executá-las com containers. Criamos imagens para o back-end e front-end, e agora avançamos para dois temas importantes: o uso completo do docker-compose e a otimização das imagens Docker.

Comecei preparando o docker-compose.yml com três serviços: MongoDB, back-end e front-end. Anteriormente, utilizávamos apenas o Mongo. A nova configuração define as imagens para cada serviço (Mongo6, jacktech/unipdi-

backend e jacktech/unipdi-frontend), seus respectivos nomes, reinícios automáticos e dependências entre eles. Também definimos variáveis de ambiente, como usuário e senha para o Mongo, e a URI de conexão no backend.

Essa estrutura permite iniciar toda a aplicação com um simples docker-compose up. Não é necessário ter IDEs ou configurar ambientes localmente. Basta possuir Docker instalado para que o compose faça o download das imagens e inicie os containers. Essa abordagem é compatível com Windows, Linux e Mac.

Ao executar o docker-compose, todos os containers foram criados e conectados corretamente em uma rede padrão. A aplicação funcionou como esperado, com persistência garantida através de volumes configurados. Ao parar e reiniciar os containers, os dados se mantiveram intactos.

Reforço a importância de dominar o docker-compose. Ele é uma ferramenta essencial para qualquer desenvolvedor que lida com múltiplos serviços, permitindo execução padronizada e compartilhamento fácil do ambiente com colegas ou em pipelines.

Na segunda parte da aula, o foco foi a otimização das imagens. Ao analisar as imagens geradas, percebi que a imagem do back-end estava com 391MB, enquanto a do front tinha apenas 52MB. Isso me levou a aplicar o conceito de **multistage build**, que permite construir imagens em etapas distintas.

O multistage build consiste em criar uma imagem intermediária para o processo de build da aplicação e outra para o ambiente de execução (runtime). Com isso, a imagem final inclui apenas o essencial para rodar a aplicação, sem ferramentas de compilação ou dependências extras.

No meu Dockerfile anterior, utilizava diretamente uma JDK completa. No novo modelo, a primeira etapa usa uma imagem do Maven com JDK para compilar o projeto. O POM e os arquivos fonte são copiados e é executado um mvn package para gerar o JAR. A segunda etapa usa apenas uma JRE para executar o JAR, resultando em uma imagem menor e mais segura.

Após construir a imagem com `docker build -t unipdi-backend-doisestagios .`, o tamanho caiu para cerca de 234MB, uma redução de mais de 150MB em relação à imagem anterior. Essa diferença é significativa, principalmente em ambientes de produção ou em pipelines de CI/CD, onde tempo e banda são recursos valiosos.

Os benefícios do multistage build são claros:

- **Imagens mais leves:** apenas o que é essencial permanece na imagem final.
- **Segurança:** menos pacotes e ferramentas expostas reduzem a superfície de ataque.
- **Desempenho:** downloads mais rápidos e execução mais eficiente.
- **Organização:** separação clara entre as fases de build e runtime.

No caso do front-end, também utilizei multistage, com a primeira etapa usando Node para construir o projeto e a segunda etapa utilizando Nginx para servir os arquivos estáticos. Isso resultou em uma imagem ainda mais leve e rápida de ser executada.

Por fim, ressalto que não subi essa imagem otimizada para o Docker Hub, mas o Dockerfile atualizado estará no repositório para que vocês possam testar. Recomendo fortemente que incluam esse tipo de abordagem em seus projetos.

Encerramos aqui o conteúdo de Docker da nossa disciplina: uso de containers, construção e distribuição de imagens, uso do docker-compose, otimização com multistage build. Na próxima aula, vamos conhecer o Podman, uma alternativa muito interessante ao Docker. Até lá!

Capítulo 5: Conhecendo o Podman

Nesta aula, seguimos nossa exploração sobre containerização, aprofundando no Podman como uma alternativa ao Docker. Antes de entrarmos no Podman em si, achei essencial compartilhar o contexto da Open Container Initiative (OCI). Trata-se de uma iniciativa da Linux Foundation em conjunto com empresas como a Docker, que visa padronizar o uso de containers. As três especificações principais da OCI são: Runtime Specification, Image Specification e Distribution Specification. Essa padronização permite que uma imagem criada em Docker possa ser executada no Podman, por exemplo, trazendo portabilidade e interoperabilidade para nossos ambientes.

A iniciativa surgiu porque, com o crescimento do uso de containers para isolar aplicações e replicar ambientes, cada ferramenta adotava um formato próprio.

Isso dificultava a portabilidade entre elas. Com as especificações da OCI, passou a ser possível criar uma imagem em Docker e executá-la em outra ferramenta como o Podman, com comportamentos padronizados.

Com esse contexto em mente, introduzi o Podman, uma ferramenta de containerização desenvolvida pela Red Hat e lançada por volta de 2019. Ele é gratuito, de código aberto e traz diferenças importantes em relação ao Docker. Uma das principais é que o Podman é "daemonless", ou seja, não depende de um processo central (daemon) para gerenciar os containers. Isso significa que, caso um processo de container falhe, apenas aquele processo será afetado, diferentemente do Docker, onde a falha no daemon pode derrubar todos os containers.

Outra diferença é que o Podman, por padrão, é "rootless". Isso permite executar containers sem precisar de permissão de superusuário, o que melhora a segurança, pois eventuais vulnerabilidades ficam restritas ao escopo do usuário comum. Embora o Docker também permita essa configuração, ela não é padrão.

Além disso, o Podman é compatível com os comandos do Docker. Ou seja, comandos como `docker run`, `docker ps` e `docker image ls` funcionam com podman da mesma forma. Para facilitar ainda mais essa transição, é possível instalar um pacote chamado `podman-docker`, que emula o uso do Docker, permitindo continuar utilizando os comandos familiares, mesmo que o ambiente esteja usando apenas Podman.

Durante a aula, demonstrei a execução de containers com o Podman, iniciando com exemplos simples, como a imagem "hello world", até exemplos mais complexos, como a execução de um banco de dados MongoDB e uma aplicação back-end feita em Java. Para isso, criei redes customizadas e volumes de dados persistentes com comandos do Podman, garantindo que os containers pudessem se comunicar e manter seus dados mesmo após o reinício.

Abordei também como construir imagens otimizadas usando um Dockerfile diretamente com Podman, aproveitando a padronização da OCI. Essa abordagem permitiu a construção de imagens em dois estágios, otimizando o tamanho final da imagem.

Outra funcionalidade importante é o suporte ao docker-compose, que pode ser utilizado com Podman através do pacote podman-compose. Mostrei como instalá-lo e executar um arquivo docker-compose.yml para subir automaticamente todos os containers necessários para uma aplicação completa, incluindo banco de dados, back-end e front-end.

Por fim, destaquei que é possível utilizar todos os recursos do Docker com o Podman, sem perder compatibilidade ou produtividade. Inclusive, fiz questão de demonstrar o ambiente 100% terminal em Linux, justamente para trazer essa experiência mais próxima da realidade de quem trabalha em ambientes de produção.

Na próxima aula, farei um overview do Podman Desktop, complementando o que vimos aqui. Até lá!

Capítulo 6: Podman Desktop

Nesta aula, finalizamos o conteúdo referente ao Podman com uma abordagem voltada para a interface gráfica oferecida pelo **Podman Desktop**. A experiência aqui é bastante diferente da linha de comando tradicional, mas igualmente poderosa e didática. A ideia foi explorar todos os recursos que essa interface oferece, facilitando o gerenciamento de containers, imagens, volumes e pods.

Comecei indicando o site oficial do Podman, o **podman.io**, onde encontramos as opções para instalar tanto o Podman CLI quanto o Podman Desktop. O site também apresenta diversas informações sobre a ferramenta: ela é leve, rápida, segura, open source e compatível com os padrões OCI. Além disso, é "Kubernetes ready", ou seja, totalmente integrada ao ecossistema Kubernetes, facilitando a criação e o deploy de pods e containers em ambientes orquestrados.

Outro ponto interessante do site é a parte de comunidade. Existe um Discord ativo, discussões abertas no GitHub, reuniões e vídeos gravados para

acompanhar as novidades. Isso mostra como a ferramenta vem sendo adotada e aprimorada em conjunto com a comunidade.

Partindo para o Podman Desktop em si, explorei os principais componentes da interface: **Dashboard**, **Containers**, **Pods**, **Imagens**, **Volumes**, **Kubernetes** e **Extensões**. No menu lateral, também encontramos a área de **Settings**, onde podemos gerenciar a versão instalada e configurar máquinas Podman (Podman Machine), que são as engines que executam os containers.

Cada Podman Machine pode ser criada com diferentes configurações de CPU, disco e memória. Podemos visualizar logs, acessar o terminal e criar quantas máquinas quisermos. O Podman Desktop também oferece integração com extensões como o kubectl (cliente de linha de comando do Kubernetes), o kind (para criar clusters locais) e o compose (para executar arquivos docker-compose). É possível configurar proxys, autenticações e repositórios de imagens, como Docker Hub e outros.

Com a interface configurada, fizemos o primeiro teste de execução de container. Criamos um container a partir da imagem hello e observamos o comportamento: por ser um container efêmero, ele é executado e finalizado rapidamente. Ainda assim, foi possível acessar os logs, realizar inspeção, ver o script correspondente em YAML e acessar um terminal (caso estivesse ativo).

Seguimos então para uma experiência mais prática: importar as imagens do MongoDB e da aplicação back-end (jacktech/unipdi-backend) diretamente pela interface. Busquei as imagens usando a funcionalidade de **pull** e, uma vez concluído o download, iniciei os containers correspondentes.

Na criação do container Mongo, especifiquei o nome, a porta (27017) e deixei os demais parâmetros padrões. Depois, fiz o mesmo com o back-end, mapeando a porta 8080. A aplicação foi executada, mas como esperado, apresentou erro de conexão com o banco de dados, já que os containers estavam isolados.

Para resolver isso, demonstrei como agrupar os dois containers em um **Pod**. Essa funcionalidade é um dos grandes diferenciais do Podman em relação ao Docker. Ao agrupar containers em um pod, eles passam a compartilhar a mesma

rede e conseguem se comunicar via localhost. Assim, o back-end passou a acessar o Mongo normalmente, validando a configuração.

Essa abordagem facilita muito o desenvolvimento e antecipa conceitos que são nativos do Kubernetes, como a comunicação entre containers no mesmo pod. Inclusive, o Podman Desktop gera automaticamente os scripts YAML correspondentes aos pods criados, que podem ser utilizados em ambientes Kubernetes.

Além disso, exploramos a aba de **Volumes**, observando que o Podman cria volumes automaticamente, e que podemos inspecioná-los para verificar onde estão sendo armazenados. As configurações de interface, como troca de tema para o modo escuro e ajustes de zoom, também estão disponíveis, contribuindo para uma experiência mais personalizável.

Finalizei a aula reforçando a importância da prática. Mesmo que o Podman Desktop ofereça uma interface amigável, o ideal é que você explore e experimente, criando containers, configurando pods e aproveitando todos os recursos da ferramenta. Isso solidifica o aprendizado e prepara para os próximos tópicos, como o Kubernetes, que veremos a seguir.

A aula serviu para concluir nossa jornada pelo universo do Podman, apresentando uma alternativa moderna e segura ao Docker, com integração completa ao ecossistema Kubernetes. Agora seguimos com mais profundidade no estudo da orquestração de containers. Até lá!

Capítulo 7: Fundamentos de Kubernetes

Nesta aula, iniciei a exploração do Kubernetes, uma ferramenta essencial para orquestração de containers. Antes de mergulharmos na parte prática, dediquei esta aula à compreensão dos conceitos fundamentais. Além disso, aproveitei para esclarecer algo importante: muitas vezes opto por uma pronúncia abrigada de termos como Kubernetes ou K8s para que ferramentas automáticas de legenda interpretem corretamente o que está sendo dito. Isso não compromete o conhecimento técnico, apenas melhora a acessibilidade.

Comecei relembrando o que vimos em Docker e Podman: os containers são poderosos para isolamento, portabilidade e replicação de ambientes, mas têm limitações quando falamos em escalabilidade, resiliência e autorrecuperação. É exatamente nesses pontos que o Kubernetes se torna essencial. Ele é um orquestrador de containers, responsável por automatizar tarefas como balanceamento de carga, recuperação automática, implantações e atualizações em larga escala. É uma ferramenta open source, que pode ser usada localmente ou em nuvem, inclusive de forma híbrida.

Expliquei, então, que antes de adotarmos qualquer solução, precisamos entender qual problema ela resolve. Quando rodamos vários containers em uma mesma máquina, essa máquina pode chegar ao limite de CPU, memória ou armazenamento. Uma opção seria melhorar essa máquina (escalabilidade vertical), mas a solução mais eficiente é a escalabilidade horizontal: distribuir a carga entre várias máquinas. O Kubernetes gerencia esse conjunto de máquinas como um cluster, distribuindo os containers conforme a disponibilidade de recursos.

Apresentei a arquitetura do Kubernetes com base em uma imagem retirada do site oficial. O cluster é composto por dois tipos de máquinas: o **control plane** (ou master node) e os **worker nodes**. O control plane é o cérebro do cluster, com quatro componentes principais:

- **API Server**: ponto de entrada para todos os comandos, geralmente acessado via kubectl.
- **Controller Manager**: garante que o estado atual do cluster esteja de acordo com o estado desejado.
- **Scheduler**: decide em qual máquina um pod será executado.
- **etcd**: banco de dados chave-valor onde está armazenado todo o estado do cluster.

Existe também o **Cloud Controller Manager**, responsável por integrações com provedores de nuvem, mas ele é opcional em ambientes locais.

Nos **worker nodes**, temos:

- **Kubelet**: comunica com o control plane, garantindo a execução dos pods.
- **Kubeproxy**: gerencia o tráfego de rede.

Ressaltei que nunca interagimos diretamente com esses componentes, sempre via API Server.

Avancei então para os objetos fundamentais:

- **Pod**: unidade mínima do Kubernetes, que pode conter um ou mais containers. Todos os containers de um pod compartilham o mesmo IP, rede e armazenamento.
- **ReplicaSet**: garante que uma quantidade definida de pods esteja sempre ativa. Se um pod falhar, outro é criado automaticamente.
- **Deployment**: gerencia ReplicaSets, versões de aplicação, atualizações e rollbacks.
- **Service**: expõe pods de forma estável, mesmo que os IPs mudem. Existem três tipos principais:
 - **ClusterIP**: acesso interno ao cluster.
 - **NodePort**: acesso externo via porta específica dos nodes.
 - **LoadBalancer**: cria um balanceador de carga externo (em provedores de nuvem).

Com esses elementos conseguimos criar uma aplicação robusta, escalável, tolerante a falhas e preparada para ambientes distribuídos. Também comentei sobre os **sidecars**, containers auxiliares em um mesmo pod, usados para tarefas como logging ou monitoramento.

Encerrei a aula ressaltando que pods isolados, chamados de **naked pods**, não são recomendados. Eles devem sempre estar vinculados a um ReplicaSet e a um Deployment. Assim garantimos resiliência, atualizações seguras e possibilidade de rollback.

Na próxima aula, vamos iniciar a parte prática, criando nosso cluster e aplicando todos esses conceitos. A documentação e materiais complementares estarão disponíveis para aprofundamento. Kubernetes é agnóstico a nuvem, então o que for aprendido aqui será válido para qualquer provedor. Vamos seguir juntos!

Capítulo 8: Criando o Cluster e a Arquitetura

Nesta aula, demos início à criação prática do nosso cluster Kubernetes utilizando o **kind** (Kubernetes in Docker). Essa ferramenta nos permite criar clusters Kubernetes dentro de containers Docker, o que é extremamente útil para ambientes de desenvolvimento e testes. Com ela, conseguimos simular toda a estrutura do Kubernetes localmente, sem precisar de provedores de nuvem.

Para começar, instalei o kind diretamente pelo site oficial e expliquei os pré-requisitos: é necessário ter o Docker instalado e em execução. A instalação do kind é simples, e o comando `kind create cluster` já cria um cluster funcional. Esse comando inicia automaticamente os containers que representarão os nodes do cluster.

Por padrão, o kind cria um cluster com apenas um node que acumula as funções de control plane e worker. No entanto, optei por uma abordagem mais realista e personalizada: criei um arquivo YAML de configuração para o kind onde defino um node do tipo control-plane e três nodes do tipo worker. Isso permite simular um ambiente de produção, em que as responsabilidades estão devidamente separadas.

Com o arquivo de configuração salvo como `kind-config.yaml`, executei o comando `kind create cluster --config kind-config.yaml --name unipdi-cluster`. O nome do cluster foi definido como `unipdi-cluster`, mantendo o padrão usado em outras partes do projeto. A criação do cluster foi rápida, e os containers foram automaticamente gerados pelo Docker, cada um representando um node.

Após a criação, usei o comando `kubectl get nodes` para listar os nodes do cluster e verificar seus status. Todos apareceram como "Ready", indicando que estavam ativos e prontos para receber pods. Também usei `kubectl get pods -A` para ver os pods do namespace `kube-system`, onde ficam os componentes internos do Kubernetes. Vimos, por exemplo, pods como o `coredns`, que cuida da resolução de nomes dentro do cluster.

Em seguida, expliquei o papel de cada componente do cluster. O `kube-apiserver`, o `kube-scheduler`, o `controller-manager` e o `etcd` rodam no control plane. Já os workers executam os pods das aplicações. Cada node também possui um agente `kubelet` e o `kube-proxy`, responsável pelo roteamento de rede. Com isso, conseguimos visualizar de forma concreta a arquitetura que estudamos na aula anterior.

Também mostrei como o `kind` cria uma rede Docker exclusiva para o cluster. Utilizei o Docker Desktop para visualizar essa rede, os containers ativos e os volumes criados automaticamente. Essa abordagem facilita muito o desenvolvimento, pois temos acesso a todas as ferramentas e podemos inspecionar cada parte do cluster com detalhes.

Um ponto importante que destaquei foi o contexto do `kubectl`. Cada vez que criamos um cluster com o `kind`, ele configura automaticamente o `kubectl` para apontar para esse novo cluster. Isso é feito atualizando o `kubeconfig`, que é o arquivo de configuração usado pelo `kubectl` para se conectar aos clusters. Podemos verificar os contextos com `kubectl config get-contexts` e alternar entre eles com `kubectl config use-context`.

Com o cluster pronto, passamos para a criação da arquitetura da aplicação. Decidi começar pelo MongoDB. Criei os manifests YAML necessários para criar o **Deployment** e o **Service** do Mongo. O Deployment define o pod com o

container da imagem oficial do Mongo e especifica as variáveis de ambiente necessárias. O Service foi criado do tipo ClusterIP, pois será usado internamente pelo back-end.

Após aplicar os arquivos com `kubectl apply -f mongo.yaml`, verifiquei os recursos criados com `kubectl get all`. O pod do Mongo iniciou corretamente, e o serviço foi registrado com um IP interno. A comunicação entre serviços no Kubernetes é feita pelo nome do serviço, então configurei a URI de conexão no back-end como `mongodb://mongo:27017/unipdi`.

Em seguida, criei o Deployment e o Service do back-end, seguindo a mesma estrutura. A imagem usada foi a `jacktech/unipdi-backend`, previamente publicada no Docker Hub. Configurei a variável de ambiente `SPRING_DATA_MONGODB_URI` para apontar para o serviço do Mongo, como mencionado anteriormente. Após aplicar os manifests com `kubectl apply -f backend.yaml`, o pod foi criado e iniciou normalmente.

Usei o comando `kubectl describe` para inspecionar os recursos, verificando logs e eventos. Com tudo funcionando, acessei a aplicação via navegador utilizando o `port-forward`, já que ainda não havíamos criado um Service do tipo `NodePort` ou `LoadBalancer`. Isso permitiu acessar a API do back-end diretamente pela porta local.

Finalizei a aula reforçando que esse processo demonstra como implantar uma aplicação real em um cluster Kubernetes, mesmo que local. Utilizamos imagens reais, serviços internos e variáveis de ambiente para configurar a comunicação entre os componentes. Na próxima aula, vamos adicionar o front-end ao cluster, completando a arquitetura completa da aplicação.

Esse conteúdo serve como base prática para qualquer implantação em nuvem. O que aprendemos aqui pode ser facilmente adaptado para AWS, Azure, GCP ou qualquer outro provedor que suporte Kubernetes. Vamos continuar avançando!

Capítulo 9: Escalabilidade e Saúde

Neste capítulo, mergulhei em dois conceitos fundamentais para garantir a estabilidade e a performance de aplicações em Kubernetes: as verificações de saúde (probes) e a escalabilidade automática de pods (Horizontal Pod Autoscaler - HPA). Meu objetivo foi mostrar, de forma prática e didática, como aplicar esses recursos no contexto real da nossa aplicação.

Comecei explicando a importância das **probes**, que são responsáveis por informar ao Kubernetes se a aplicação está ativa (liveness) e pronta para receber requisições (readiness). Mesmo que o pod esteja ativo, isso não significa que a aplicação dentro dele esteja funcionando corretamente ou pronta para operar. Por isso, essas checagens são essenciais.

A primeira probe que configurei foi a **liveness probe**, que atua como uma “prova de vida”. Ela realiza requisições HTTP periódicas para o container e, caso obtenha um status diferente de sucesso (200–299), após três tentativas, o Kubernetes entende que o container está com problemas e o reinicia. Configurei essa verificação com um `initialDelaySeconds` de 20, um `periodSeconds` de 10 e um `failureThreshold` de 3. Isso significa que a aplicação tem 20 segundos para iniciar antes da primeira verificação, que ocorre a cada 10 segundos, e após três falhas consecutivas o container é reiniciado.

A segunda foi a **readiness probe**, que verifica se a aplicação está pronta para receber tráfego. Embora parecida com a liveness, ela não reinicia o container em

caso de falhas. Em vez disso, o pod é marcado como “not ready” e deixa de receber requisições. Essa configuração evita o envio de tráfego para aplicações que ainda estão carregando scripts ou conexões iniciais. Usei os mesmos parâmetros da liveness, mas aqui o impacto é diferente: sem reinício, apenas retirada do tráfego até que a aplicação esteja pronta.

Durante essa configuração, o Kubernetes emitiu alertas informando que os containers não tinham definições de recursos. Isso me levou à próxima parte da aula: a definição de **resources**, tanto limits quanto requests. Essa etapa é crucial para evitar sobrecargas, pois permite ao Kubernetes entender quanto de CPU e memória cada container pode usar, garantindo isolamento e previsibilidade de consumo.

A partir dessas definições, avançamos para o tema principal deste capítulo: **escalabilidade horizontal com HPA**.

O **Horizontal Pod Autoscaler (HPA)** é o recurso do Kubernetes responsável por escalar a quantidade de pods com base na utilização de recursos, como CPU e memória. Configurei o HPA para monitorar o front-end da nossa aplicação. Especificamente, determinei que, se a utilização de CPU ultrapassasse 50%, o número de pods deveria ser ajustado entre um mínimo de dois e um máximo de dez.

Para que o HPA funcione corretamente, é necessário um **servidor de métricas**. Baixei e configurei o metrics-server, necessário para coletar dados de uso em tempo real. Um detalhe importante foi a inclusão do argumento `--kubelet-insecure-tls`, indispensável para que o metrics-server funcione em ambientes locais como o nosso.

Depois de aplicar essa configuração, usei o comando `kubectl get hpa` para visualizar o estado do HPA. No início, o uso de CPU era baixo, mas, para simular um cenário de alta demanda, ajustei os valores de limits e requests para CPU com valores baixos (por exemplo, 1%). Com isso, o Kubernetes detectou uma sobrecarga e começou a criar novas réplicas do pod automaticamente. Em poucos minutos, as réplicas aumentaram até o limite configurado de dez pods.

Esse experimento mostrou como o Kubernetes responde dinamicamente ao aumento de carga, garantindo resiliência e disponibilidade. A capacidade de escalar automaticamente com base em métricas reais é um dos grandes diferenciais da orquestração com Kubernetes.

Fechei a aula reforçando que esses conceitos de probes e HPA são essenciais em ambientes reais. Apesar de parecerem simples, seu impacto na estabilidade e performance da aplicação é enorme. A partir de agora, toda vez que desenvolvermos uma aplicação para rodar em Kubernetes, devemos considerar essas configurações desde o início do desenvolvimento.

Por fim, deixei os arquivos atualizados disponíveis em uma branch específica (aula 9), incluindo os manifests configurados, o arquivo components.yaml do metrics-server e todas as alterações feitas. O convite que faço é para que você experimente no seu ambiente, altere os parâmetros, monitore os resultados e consulte a documentação oficial. Com prática e curiosidade, esses conhecimentos se tornam naturais e aplicáveis em qualquer projeto corporativo.

Na próxima aula, vamos iniciar nossos estudos com AWS, mergulhando em novos conceitos e ferramentas. Até lá!

Capítulo 10: Fundamentos AWS

Chegamos a uma etapa fundamental do nosso curso: a integração dos nossos serviços com a nuvem. Esse é um requisito indispensável nos dias de hoje. Saber replicar o que fazemos localmente em um ambiente de nuvem é essencial para qualquer desenvolvedora ou desenvolvedor. E, neste curso, escolhemos trabalhar com a AWS (Amazon Web Services).

A AWS é uma das maiores plataformas de computação em nuvem do mundo, lançada em 2006. Ela se tornou referência por oferecer mais de 200 serviços que atendem a praticamente qualquer necessidade de desenvolvimento e infraestrutura. Seus datacenters estão distribuídos globalmente, com cerca de 100 zonas de disponibilidade em 33 regiões. A escolha da região de hospedagem, inclusive, é um ponto importante que detalharei mais adiante.

Comecei essa aula apresentando os principais serviços da AWS que utilizamos no dia a dia como profissionais de desenvolvimento. Dentro da categoria **Computação**, destaquei:

- **EC2 (Elastic Compute Cloud):** permite criar e gerenciar máquinas virtuais com controle total sobre o sistema operacional, runtime, segurança, e configurações da aplicação. No entanto, toda a manutenção e configuração ficam por nossa conta.
- **ECS (Elastic Container Service):** é a solução de orquestração de containers da AWS. Ele executa containers, aloca recursos automaticamente, faz balanceamento de carga e escalabilidade, com menos responsabilidade do desenvolvedor sobre a infraestrutura.

- **Lambda**: é a proposta serverless da AWS. Permite executar código sem gerenciar servidores, pagando apenas pelo tempo de execução. Ideal para microsserviços, integrações ou eventos com baixa frequência.
- **EKS (Elastic Kubernetes Service)**: é o serviço de Kubernetes gerenciado, que possibilita rodar nossos clusters com infraestrutura provisionada pela AWS.

Na área de **Armazenamento**, destaquei o **S3 (Simple Storage Service)**, que utilizamos para armazenar arquivos como imagens, vídeos e textos. Vamos fazer uma implementação prática com ele nas próximas aulas.

Falando de **banco de dados**, os principais serviços abordados foram:

- **RDS (Relational Database Service)**: banco relacional compatível com MySQL, PostgreSQL, MariaDB, e Aurora. Este último é o banco relacional nativo da AWS, com alta performance e compatibilidade com MySQL e PostgreSQL.
- **DynamoDB**: banco de dados NoSQL, totalmente gerenciado, com baixa latência. Também será utilizado nas práticas deste módulo.

Abordei ainda a parte de **redes**, essencial para garantir a performance e segurança da aplicação:

- **VPC (Virtual Private Cloud)**: cria uma rede isolada para os serviços, aumentando a segurança e o controle.
- **Gateway e Load Balancer**: usados para gerenciar tráfego, centralizar requisições e distribuir carga.

Na área de **segurança**, o destaque foi para o **IAM (Identity and Access Management)**. Esse serviço permite gerenciar usuários, permissões, grupos e políticas. Com ele, definimos o que cada pessoa ou aplicação pode acessar. No ambiente corporativo, isso garante que as permissões estejam adequadas ao papel de cada um — como desenvolvedores, QAs, DevOps, etc. Um recurso poderoso do IAM é a possibilidade de configurar papéis e políticas específicas para aplicações também, não apenas usuários.

Expliquei também o modelo de cobrança da AWS, baseado no conceito de **"pay as you go"**, ou seja, você paga conforme usa. No entanto, destaquei uma pegadinha importante: se você criar uma instância e não usá-la, ela ainda assim gerará custo, pois está disponível para uso. Por isso, é fundamental acompanhar os recursos ativos e deletar aquilo que não está mais sendo utilizado. Recomendei configurar alertas de custo para evitar surpresas desagradáveis.

Em seguida, destaquei o papel das **regiões e zonas de disponibilidade**. Quando falamos que algo está "na nuvem", na verdade, estamos nos referindo a um datacenter físico em algum lugar do mundo. A **latência**, ou seja, o tempo de resposta da aplicação, depende da distância entre os usuários e o local de hospedagem. Por isso, ao escolher a região, é importante considerar onde estão os usuários. Hospedar em São Paulo, por exemplo, pode oferecer baixa latência para usuários brasileiros, embora nem sempre seja a opção mais barata. Em algumas empresas, a decisão entre latência e custo será tomada por arquitetos ou líderes de DevOps, com base nas necessidades do negócio.

As **zonas de disponibilidade (AZs)** são responsáveis pela redundância da aplicação. Se uma zona falhar, outra pode assumir, garantindo a continuidade da operação. Essa é uma estratégia de alta disponibilidade que reforça a confiabilidade da infraestrutura da AWS.

Por fim, demonstrei o processo de criação de uma conta na AWS. Mostrei o preenchimento dos dados, a escolha do plano gratuito (que oferece 200 dólares em créditos por seis meses), e destaquei a obrigatoriedade de informar um cartão de crédito, mesmo para contas gratuitas. A AWS realiza uma cobrança temporária de verificação, que é estornada em seguida.

Além disso, introduzi o **AWS CLI (Command Line Interface)**, que permite interagir com os serviços da AWS diretamente pela linha de comando. Mostrei onde encontrar a documentação de instalação e como verificar a versão instalada. Ressaltei que, ao longo do curso, utilizaremos tanto o console quanto o CLI, então é importante tê-lo instalado e configurado.

Finalizei com uma orientação importante: **nós, professores da UNIPDS, não nos responsabilizamos por eventuais cobranças geradas no uso da AWS.**

Toda a infraestrutura usada no curso é baseada em ferramentas gratuitas. Cabe a cada aluno gerenciar seus recursos e verificar os custos com responsabilidade. Use, teste, aprenda e, em seguida, apague os recursos que não forem mais necessários.

Na próxima aula, começaremos com nossa primeira integração prática: vamos armazenar dados em um bucket S3 diretamente a partir de uma aplicação Java. Então, deixe sua conta pronta e CLI instalado. Até a próxima!

Capítulo 11: Integração com AWS S3

Nesta aula, o objetivo foi realizar a integração da nossa aplicação Java com o serviço de armazenamento S3 da AWS. O Amazon S3 (Simple Storage Service) é uma solução altamente escalável, segura e de baixa latência, que permite armazenar qualquer volume de dados de maneira confiável. Cada arquivo armazenado no S3 é considerado um objeto, e esses objetos ficam organizados em buckets, que funcionam como contêineres de armazenamento.

Para esse exemplo prático, decidimos estender a funcionalidade da aplicação adicionando a opção de upload de currículos pelos usuários. A proposta é que, além de gerenciar os dados do PDI, a pessoa usuária possa anexar um currículo em PDF que será armazenado diretamente em um bucket na AWS.

O primeiro passo foi a criação de um bucket na AWS, localizado na região de São Paulo. Reforcei a importância de assistir aos vídeos e consultar a documentação oficial da AWS, que são extremamente ricos em detalhes e facilitam bastante o entendimento dos recursos. Para criar o bucket, escolhi o nome "unipdi" — um nome único globalmente no namespace do S3. Isso significa que não pode haver dois buckets com o mesmo nome em toda a AWS, independentemente do usuário ou da conta.

Durante a configuração, mantive as ACLs desabilitadas e optei por bloquear todo o acesso público inicialmente. Esse bloqueio é revertido mais tarde por meio da criação de uma política personalizada de acesso, garantindo que os dados armazenados estejam protegidos, mas acessíveis quando necessário.

Na aplicação, fui até o IntelliJ e interrompi a execução da aplicação para configurar as dependências necessárias. No pom.xml, adicionei a dependência do spring-cloud-starter-aws-s3, que nos fornece as classes e interfaces para interagir com o S3. Após o reload das dependências, configurei as credenciais e a região da AWS no arquivo application.properties, utilizando variáveis de

ambiente: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, a região (como `sa-east-1`) e o nome do bucket.

Para obter essas credenciais, acessei o console da AWS, na seção de gerenciamento de credenciais, e criei uma nova chave de acesso. Reforcei que essas credenciais devem ser protegidas e que o ideal é utilizá-las apenas em ambientes locais para testes. No IntelliJ, configurei essas variáveis diretamente nas configurações de execução do projeto.

Com as configurações feitas, criei a classe `CurriculoService`, responsável pela lógica de upload dos arquivos. Essa classe utilizou dois componentes principais: o `S3Client`, para enviar os arquivos ao bucket, e o `S3Presigner`, para gerar URLs temporárias que permitem o acesso seguro aos arquivos.

O método de upload salvava o arquivo em uma pasta chamada `curriculo`, dentro do bucket, nomeando cada arquivo com a matrícula da pessoa, por exemplo: `curriculo/12345.pdf`. Já o método de geração de URL temporária permitia especificar por quantos minutos o link ficaria disponível. Esse tempo é essencial para garantir que apenas o usuário tenha acesso momentâneo ao seu currículo.

Na sequência, implementei a `CurriculoController`, um `RestController` com um endpoint para upload e geração da URL. Para isso, também foi necessário adicionar o atributo `curriculoUrl` na entidade `Pessoa` e propagá-lo até o DTO de resposta. Essa URL era armazenada no banco para indicar visualmente se um currículo já foi enviado para determinada pessoa.

Demonstrei todo o fluxo: com a aplicação em execução, criei uma nova pessoa, selecionei um arquivo PDF e cliquei em enviar. O back-end gerou a URL temporária e retornou ao front-end. Ao clicar no link, a aplicação redirecionava para o conteúdo armazenado no S3 com sucesso.

No console da AWS, validei que os arquivos haviam sido armazenados corretamente no bucket. A organização por matrícula e a estrutura de pastas no bucket seguiram exatamente como configurado. Testei também com outros usuários para garantir que o sistema fosse escalável e mantivesse o comportamento desejado.

Finalizei reforçando alguns pontos importantes:

- O nome do bucket deve ser único em toda a AWS.
- A região do bucket precisa ser especificada na aplicação para garantir a conectividade.
- O armazenamento no S3 possui diferentes classes (tiers), cada uma com custos distintos, e o padrão utilizado foi o STANDARD.
- A geração de URLs temporárias garante segurança e controle de acesso.

Além disso, comentei que futuramente vamos usar esse mesmo fluxo para disparar notificações via AWS Lambda sempre que um novo currículo for enviado. Essa integração permitirá ações automáticas a partir de eventos, agregando ainda mais valor à nossa aplicação.

Por fim, deixei o código atualizado disponível em uma branch específica e recomendei que os alunos façam testes práticos com suas próprias contas na AWS, reforçando que a prática é essencial para consolidar esse tipo de conhecimento técnico.

Na próxima aula, vamos trabalhar com o DynamoDB, banco NoSQL da AWS, continuando a evolução da nossa aplicação na nuvem.

Capítulo 12: Armazenando Dados com DynamoDB

Neste capítulo, explorei a integração da nossa aplicação Java com o Amazon DynamoDB, um banco de dados NoSQL da AWS. Essa etapa marca mais um passo importante na evolução da aplicação, agora com armazenamento de dados escalável e de altíssimo desempenho.

Antes de iniciar a prática, reforcei um conceito fundamental: o aprendizado sólido acontece quando aplicamos novos recursos a projetos reais e em constante crescimento. Foi exatamente assim que desenhei nossa jornada. Começamos com MongoDB, depois passamos a integrar com o S3, e agora avançamos para o DynamoDB. A aplicação ganha robustez e uma arquitetura mais corporativa a cada nova etapa.

O DynamoDB é um banco NoSQL criado pela Amazon para lidar com grandes volumes de dados com baixíssima latência. Sua promessa é impressionante: menos de 10 milissegundos de resposta em qualquer escala. É um banco orientado a documentos, semelhante ao MongoDB, utilizando estruturas de chave-valor. No entanto, ao contrário do Mongo, que usa coleções e documentos, o DynamoDB organiza seus dados em **tabelas**, e cada item da tabela é um conjunto de atributos.

Um dos pontos que destaquei é que o DynamoDB não foi feito para joins ou consultas complexas, como ocorre em bancos relacionais. Seu ponto forte é a escalabilidade automática e a performance em consultas simples, especialmente quando usamos corretamente a **chave de partição**.

Para esta integração, refatorei completamente a estrutura da aplicação. Ao contrário do modelo anterior, que separava Pessoa e PDI em coleções distintas no MongoDB, aqui tudo foi consolidado em uma única tabela Pessoa, contendo todos os PDIs e suas respectivas metas como listas aninhadas. Essa estrutura é mais adequada ao modelo do DynamoDB, permitindo acessos rápidos e organizados por uma única chave de partição.

A chave de partição escolhida foi a **matrícula**, pois já havíamos utilizado esse campo como referência principal em outras funcionalidades da aplicação. Essa chave precisa ser única e presente em todas as interações com o banco.

Na prática, criei a tabela diretamente no console da AWS, definindo o nome Pessoa (com letra maiúscula, respeitando a convenção da nossa classe) e atribuindo matrícula como chave de partição. Não utilizei chave de classificação nesta etapa, mas comentei que ela pode ser útil em cenários que exigem consultas com base em mais de um critério.

No projeto Java, removi completamente o pacote Repository, pois agora usamos o DynamoDbEnhancedClient, que nos fornece o template para salvar, buscar e listar dados. A classe Pessoa foi anotada com `@DynamoDbBean`, e criei uma configuração personalizada (TableNameResolver) para garantir que a tabela correta fosse reconhecida com segurança pelo client.

Além disso, adaptei os getters e setters para incluir as anotações `@DynamoDbPartitionKey` e `@DynamoDbAttribute`, deixando claro para o framework como serializar e persistir cada campo. Também precisei anotar a classe PDI com `@DynamoDbBean`, mesmo ela sendo um atributo aninhado, para garantir a correta serialização da lista.

Outro ponto importante foi a alteração nos DTOs, removendo campos desnecessários como IDs, e ajustando os dados retornados. Também criei um ExceptionHandler para lidar com tentativas de cadastro de matrículas duplicadas, já que a chave de partição deve ser única. Essa validação ocorre diretamente no serviço antes da gravação dos dados.

O serviço passou por grandes mudanças. Agora, usamos o DynamoDbTemplate para salvar e buscar dados. Para localizar uma pessoa, montamos uma QueryConditional com base na matrícula e utilizamos a API query() do template. Para salvar, usamos o método save(), que também serve para atualizações. Ressaltei que, apesar de termos um método listarTodos, seu uso em produção deve ser criterioso, pois a operação scan() percorre todos os itens e pode ser custosa em bases com milhões de registros.

No teste da aplicação, cadastrei uma nova pessoa com matrícula e nome, e confirmei no console da AWS que os dados foram armazenados corretamente. Em seguida, adicionei um PDI com duas metas, e observei como essa estrutura foi salva em formato aninhado no DynamoDB. O item continha a matrícula, o nome, e uma lista de PDIs, cada um com suas metas também em formato de lista. A visualização da estrutura no console deixou claro como os dados estão organizados e como o acesso a essas informações é direto e eficiente.

Aproveitei o momento para destacar recursos adicionais do DynamoDB, como a possibilidade de criar **índices secundários**, **réplicas globais** e **criptografia** com chaves gerenciadas pela AWS. Também abordei a precificação, que pode seguir dois modelos:

1. **Capacidade sob demanda:** você paga apenas pelo uso, ideal para cenários imprevisíveis ou aplicações que crescem organicamente.
2. **Capacidade provisionada:** define previamente o número de leituras e gravações por segundo, garantindo estabilidade e performance previsível.

Ambos os modelos podem ser combinados com recursos como auto scaling, backups automáticos e monitoramento por meio do CloudWatch. Aconselhei fortemente a criação de alarmes e limites de custo para evitar surpresas na fatura da AWS.

Fechei o capítulo reforçando que toda essa implementação está disponível em uma branch específica no GitHub. O código está pronto para ser utilizado como base em projetos reais, e minha recomendação é que os alunos pratiquem criando suas próprias tabelas, adaptando a modelagem e observando como o DynamoDB responde a diferentes estruturas.

Na próxima aula, daremos continuidade à nossa jornada com a AWS, implementando uma função Lambda para reagir ao envio de currículos. Essa será mais uma peça na arquitetura moderna que estamos construindo. Até lá!

Capítulo 13: AWS Lambda

Nesta aula, avançamos mais um passo na integração da nossa aplicação com os serviços da AWS, implementando uma função Lambda responsável por enviar notificações via e-mail sempre que um novo currículo for enviado para o S3.

Expliquei inicialmente o conceito de *Lambda Functions*, que são funções serverless da AWS. Ao contrário das EC2, onde precisamos gerenciar toda a infraestrutura (máquina virtual, sistema operacional, rede, etc.), no modelo serverless, a função é executada sob demanda, e a AWS cuida de toda a infraestrutura. Isso significa que não nos preocupamos com instâncias ociosas nem com escalabilidade manual: pagamos apenas pelos milissegundos de execução da função.

A ideia da nossa Lambda era simples, mas poderosa: toda vez que um currículo fosse enviado ao S3, essa função seria disparada automaticamente, capturando o evento e enviando uma notificação por e-mail para o RH. Embora fosse possível utilizar o Amazon SNS para isso, escolhi criar uma aplicação Java encapsulada em uma função Lambda para mostrar, na prática, como fazer essa integração com nosso próprio código.

Criei um novo projeto Java chamado unipdi-notificacao, utilizando o Java 21. Nele, adicionei as dependências necessárias:

- **AWS SDK S3**: para que a função possa processar eventos vindos do S3;
- **AWS SDK SSM (System Manager)**: para acessar os parâmetros armazenados na Parameter Store (como remetente e destinatário do e-mail);
- **Spring Boot Starter Mail**: para realizar o envio dos e-mails com uma configuração SMTP simples.

O projeto foi mantido extremamente enxuto. Excluí a classe main e criei a classe NotificacaoHandler, que implementa a interface RequestHandler<S3Event, String>. Essa interface exige a implementação do método handleRequest, que é chamado sempre que a função for invocada.

No construtor da classe, inicializei o `SSMClient` da AWS, permitindo buscar parâmetros da Parameter Store. Esses parâmetros foram fundamentais para evitar deixar configurações sensíveis no código-fonte, como e-mails e senhas. Com isso, conseguimos configurar, via console da AWS, o e-mail remetente, destinatário e a senha de aplicação para o envio do e-mail.

Dentro do método `handleRequest`, fiz a extração dos dados do evento do S3. O evento traz um JSON com os detalhes do objeto recém-enviado ao bucket. Dele, capturei o nome do bucket e a chave do objeto, que corresponde ao nome do arquivo (currículo). Utilizei essas informações para compor o conteúdo do e-mail.

Montei um `SimpleMailMessage`, utilizando o `JavaMailSenderImpl`, já que não estávamos usando o Spring por completo. Configurei manualmente as propriedades SMTP necessárias, como autenticação, `startTLS`, host e porta, diretamente na aplicação.

Com a mensagem montada, o envio foi feito. Em caso de sucesso, loguei a confirmação; em caso de falha, capturei a exceção e retornei uma mensagem de erro. Essa estrutura nos permitiu visualizar os resultados da execução no CloudWatch Logs, onde analisamos as execuções e seus respectivos outputs.

Para que a função pudesse ser executada corretamente, foi necessário gerar um `.jar` da aplicação via Maven. Esse arquivo foi então carregado manualmente no console da AWS Lambda, substituindo o código de exemplo padrão. Também foi necessário indicar corretamente o manipulador da função, no formato `br.com.unipds.NotificacaoHandler::handleRequest`, garantindo que a Lambda soubesse qual classe executar.

Após o deploy, configurei o gatilho da função, associando o bucket S3 como origem de eventos. Especificamente, escolhi eventos de criação de objetos. Isso significa que toda vez que um novo arquivo fosse enviado ao bucket unipdi, a Lambda seria acionada.

Criei também os parâmetros na Parameter Store da AWS, nos seguintes caminhos:

- `/notificacao/email/user`: remetente do e-mail

- /notificacao/email/password: senha de aplicação do e-mail remetente
- /app/email/rh: destinatário da notificação

Para gerar a senha de aplicação, mostrei o processo pelo Gmail: acessar "Gerenciar sua Conta do Google", navegar até "Segurança", localizar "Senhas de app" e gerar uma nova senha vinculada ao remetente. Essa senha foi usada no parâmetro password para autenticação SMTP.

O primeiro teste resultou em erro por falta de permissão de leitura na Parameter Store. Corrigi isso adicionando à função Lambda as permissões necessárias via IAM:

- **AmazonSSMReadOnlyAccess**
- Uma política customizada em linha com permissões para ssm:GetParameter e ssm:GetParametersByPath.

Após ajustar as permissões, novos testes mostraram que a função funcionava corretamente. Cada envio de currículo ao S3 gerava um e-mail para o RH, com as informações detalhadas do arquivo e do bucket, validando a integração end-to-end da nossa aplicação com os serviços AWS.

Finalizei explicando a diferença entre Parameter Store e Secrets Manager. Embora ambos possam armazenar dados sensíveis, o Parameter Store é ideal para configurações simples, com custo zero dentro do tier gratuito. Já o Secrets Manager é indicado para dados críticos como tokens e credenciais rotativas, oferecendo auditoria e maior controle, porém com custos adicionais.

Com isso, encerramos a parte prática da integração com Lambda. Nossa aplicação agora é capaz de reagir automaticamente a eventos e tomar ações sem depender de servidores, o que representa um avanço significativo rumo a uma arquitetura moderna, escalável e orientada a eventos.

Na próxima aula, entraremos no mundo do CI/CD, com foco em automação de deploys e pipelines. Estou empolgada para mostrar os próximos passos. Até lá!

Capítulo 14: Integração e Entrega Contínua com Github Actions

Antes de mergulharmos na implementação de CI/CD com GitHub Actions, quero relembrar algo importante sobre custos na AWS. Durante nossas últimas aulas, criamos diversos recursos como Bucket S3, AWS Lambda e DynamoDB. Mesmo que alguns estejam parados, eles podem gerar cobranças se estiverem fora do free tier. Por isso, sempre configuro alertas de orçamento para evitar surpresas. É simples: no console da AWS, busco por "gerenciamento de cobranças e custos", onde consigo ver os recursos utilizados e criar orçamentos com alertas, por exemplo, para me notificar quando os gastos passarem de US\$1.

Agora, vamos ao tema principal: integração e entrega contínua com GitHub Actions. GitHub Actions é a ferramenta de CI/CD nativa do GitHub, que permite criar fluxos de trabalho automatizados. O objetivo é reduzir tarefas manuais e tornar o desenvolvimento mais eficiente. Ao invés de eu, por exemplo, gerar o .jar da Lambda manualmente, subir no console e configurar tudo na mão, quero que isso ocorra de forma automática.

No GitHub, cada repositório possui uma aba chamada "Actions". Lá posso configurar scripts que são executados automaticamente quando determinados eventos acontecem, como um push ou um pull request. Isso é similar aos gatilhos que usamos na AWS Lambda, como o upload de arquivos no S3 que dispara a execução da função.

Para começar, criei uma branch a partir da anterior, e dentro do meu repositório unipdi (onde estão todos os projetos organizados por pastas), adicionei uma nova pasta chamada .github/workflows. É nesse local que crio os arquivos YAML que definem os fluxos de trabalho. O nome do arquivo YAML foi deploy-lambda-notificacao.yaml e ele será responsável por fazer build e deploy da função Lambda automaticamente sempre que eu der um push no repositório.

O YAML começa com um name, definindo o nome do workflow, e um on, que determina qual evento deve disparar a execução — nesse caso, qualquer push. Em seguida, defino os jobs, que são os blocos de tarefas a serem realizadas, e cada job contém vários steps, que são os passos individuais do processo.

O primeiro passo foi o checkout do repositório usando a action actions/checkout@v3. Depois, configurei o ambiente Java com actions/setup-

java@v3, especificando a versão 21 e a distribuição Temurin, que é a mesma usada nas imagens Docker que criamos anteriormente.

Com o ambiente configurado, o próximo passo foi realizar o build com Maven, usando `mvn clean package -DskipTests` para gerar o `.jar` da aplicação. Após isso, renomeei o arquivo `.jar` para algo mais legível, usando o comando `mv target/unipdi-*-SNAPSHOT.jar target/unipdi-notificacao.jar`, garantindo que o nome estivesse limpo e padrão.

Na sequência, configurei as credenciais da AWS usando a action `aws-actions/configure-aws-credentials@v4`, passando `aws-access-key-id`, `aws-secret-access-key` e a região `sa-east-1`. Essas credenciais foram previamente salvas nos Secrets do repositório GitHub, em Settings > Secrets and Variables > Actions.

Por fim, usei um comando `aws lambda update-function-code` para atualizar a função Lambda com o novo `.jar`, garantindo o deploy automático da aplicação. Claro, como toda pipeline real, enfrentei diversos erros: digitação errada, configurações faltando, diretórios incorretos. Corrigi cada um deles passo a passo, como acontece na vida real.

Testei a pipeline criando um novo push com ajustes na função e validando o recebimento da notificação no meu e-mail. A mensagem configurada apareceu corretamente, indicando que o fluxo funcionou como esperado.

Com isso, qualquer alteração futura no código da minha aplicação de notificação será automaticamente buildada e publicada na AWS, sem necessidade de repetir todos os passos manualmente. GitHub Actions oferece ainda a possibilidade de criar múltiplos workflows para diferentes situações: merge, criação de branches, publicação, etc. Além disso, o GitHub Marketplace possui diversas actions prontas que podemos aproveitar.

Fico feliz de mostrar essa implementação na prática, mesmo com os erros e ajustes ao longo do caminho. Isso faz parte do aprendizado. Na próxima aula, vamos continuar explorando CI/CD, mas utilizando outra ferramenta. Até lá!

Capítulo 15: Pipelines Declarativas com Jenkins

Dando continuidade ao nosso estudo de CI/CD, agora é hora de explorarmos uma das ferramentas mais utilizadas no mercado: o Jenkins. Antes, revisando os conceitos, CI (Continuous Integration) refere-se à prática de compilar e testar o código automaticamente a cada commit, enquanto CD (Continuous

Deployment) trata da entrega automatizada em ambientes como teste, homologação e produção.

O Jenkins surgiu em 2004 com o nome Hudson, tornando-se Jenkins em 2011. É um servidor de automação de código aberto, desenvolvido em Java, e surgiu da necessidade de um desenvolvedor que queria garantir que seu código estava correto antes de fazer o commit na branch principal. O sucesso da ferramenta está justamente no fato de ela resolver um problema real de forma eficiente — um exemplo perfeito de alguém que amou o problema e construiu uma solução robusta.

Assim como o GitHub Actions, o Jenkins permite integrações com diversas ferramentas por meio de plugins. A comunidade é responsável pela maior parte desses plugins, o que garante uma grande variedade de opções, mas também exige atenção: muitos plugins podem se tornar obsoletos ou serem descontinuados.

Uma característica importante do Jenkins é sua arquitetura composta por um Jenkins Server e os Jenkins Nodes (ou Agents). O server é onde configuramos as pipelines e as automações, enquanto os agentes são responsáveis por executar essas automações. Ao instalar o Jenkins no Windows, por exemplo, um agente padrão já é incluído.

Na instalação, utilizamos a versão LTS (Long Term Support) para maior estabilidade. A ferramenta pode ser executada em vários ambientes — Windows, Docker, Kubernetes, Fedora — mas como estamos em um laboratório local, escolhi o Windows. Durante a instalação, vale lembrar que o Jenkins usa por padrão a porta 8080, o que pode gerar conflitos com aplicações Spring Boot. Felizmente, podemos mudar essa configuração no arquivo `jenkins.xml`.

Após a instalação, o Jenkins exige a senha de desbloqueio, que é gerada em um arquivo local. Em seguida, criamos um usuário administrador e realizamos as configurações iniciais. Uma delas é a definição da URL do servidor Jenkins. Por fim, instalamos os plugins sugeridos automaticamente.

Com o Jenkins em funcionamento, iniciei a criação de uma pipeline de teste. Escolhi a opção "pipeline" ao criar uma nova tarefa, inseri uma breve descrição e optei por não aceitar builds concorrentes, garantindo que apenas uma execução ocorra por vez. No editor de script da pipeline, usei a sintaxe declarativa do Jenkins para construir um simples "Hello World". Isso envolveu a definição de pipeline, agent, stages e steps, de forma semelhante aos jobs e steps do GitHub Actions. Após salvar e executar, conferi no console output o resultado da execução com sucesso.

Com a pipeline de teste validada, passei à criação da pipeline real do projeto UniPDI Notificação, escolhendo o tipo "multi-branch pipeline", pois o repositório contém várias branches. Informei que o código-fonte está no GitHub e adicionei credenciais de acesso por meio de um Personal Access Token (PAT). Essas credenciais foram cadastradas diretamente no Jenkins, com ID git.creds, e associadas ao meu usuário no GitHub.

Configurei o Jenkins para descobrir todas as branches do repositório e utilizei a opção "Build Configuration by Jenkinsfile". Para isso, criei o arquivo Jenkinsfile dentro do repositório, substituindo a antiga pasta .github/workflows. Esse Jenkinsfile define a pipeline declarativa com os seguintes estágios:

1. **Checkout:** Clonagem do repositório.
2. **Build com Maven:** Execução do mvn clean package dentro do diretório unipdi-notificacao.
3. **Renomeação do .jar:** Padronização do nome do arquivo gerado.
4. **Deploy na AWS Lambda:** Atualização automática da função Lambda utilizando credenciais AWS armazenadas no Jenkins com o ID aws-creds.

As credenciais AWS foram configuradas no Jenkins como usuário e senha, representando respectivamente o aws-access-key-id e o aws-secret-access-key. Após configurar tudo, forcei um scan do repositório, e o Jenkins detectou a nova branch com o Jenkinsfile. Ao executar a pipeline, inicialmente houve falha por falta das credenciais AWS, mas, uma vez cadastradas corretamente, a execução foi bem-sucedida.

No console output, pude verificar todas as etapas, incluindo o sucesso na atualização da função Lambda, com exibição do ARN e da role utilizados. O Jenkins provou ser uma ferramenta poderosa, permitindo execuções sob demanda e muito controle sobre as pipelines.

Essa abordagem amplia nossas opções para CI/CD, mostrando que não devemos nos limitar ao GitHub Actions. O Jenkins oferece uma flexibilidade tremenda, embora exija maior responsabilidade na manutenção e configuração.

Encerramos aqui a nossa disciplina, que percorreu uma ampla gama de temas: Docker, Podman, Kubernetes, fundamentos de cloud com AWS, e finalmente CI/CD com GitHub Actions e Jenkins. Espero que o conteúdo tenha sido valioso e aplicável à sua jornada profissional. Agora, é hora de você colocar tudo isso em prática. O aprendizado só se consolida verdadeiramente quando fazemos. Até a próxima!

